

# 洛谷 AI 学习测评报告

Coach Edition · 图表化诊断 / 教练反馈 / 训练规划

## 测评基础信息

选手姓名：黄鼎

测评时间：2026-06-03 13:23

所属学校：深圳实验

当前年级：高二

已通过题数

438

未通过题数

0

平均难度

2.8

高频标签

暂无

## 报告目录

---

1. 核心数据概览与图表化分析
2. 综合能力雷达表与诊断
3. 考纲精准定级与知识点盲区
4. 风险诊断与训练闭环表
5. 代码质量与工程习惯
6. 定制训练题单
7. 错题单页题解与复盘（含 AI 题解摘要与推荐同类题）

# 1. 核心数据概览与图表化分析

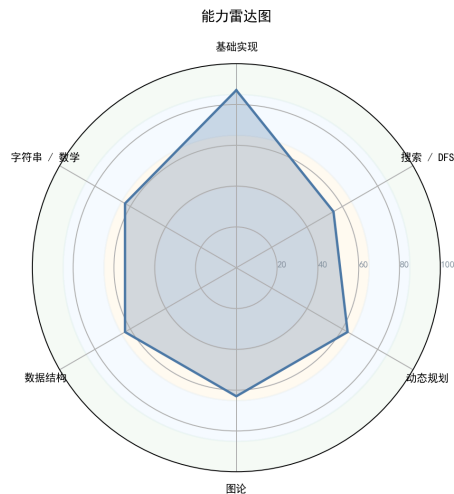


图 1: 能力雷达图

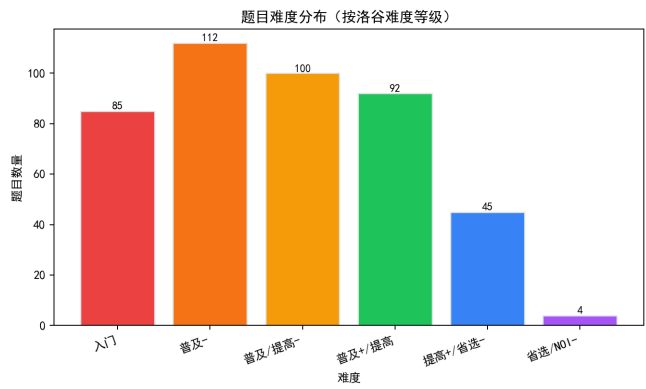


图 2: 题目难度分布

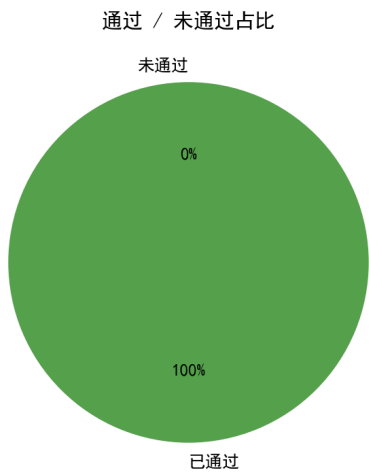


图 3: 通过 / 未通过占比

图 4: 高频标签分布

好的，教练在此。这份记录相当干净，438题全通过，没有卡住的题，这本身就是一种实力的体现。但这不代表没有提升空间，反而意味着我们需要更精准地挖掘深层问题。我将基于你提供的所有材料，进行一次深度诊断与战略规划。

### 选手深度诊断与突破辅导报告

#### 1. 【综合能力雷达表与诊断】

| 能力块       | 当前等级   | 数据证据                                                                                  | 已经具备                                                                   |
|-----------|--------|---------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| 基础实现/代码落地 | 稳      | 全局通过 438 题，难度 1-4 题占比 88.8%。代码结构清晰， <code>main</code> 函数逻辑清楚。                         | 能够将任何明确算法思路稳定、快速地转化为可运行且无低级错误的 C++ 代码。                                 |
| 高级算法/数据结构 | 强项但需精炼 | 通过了 P3195 (玩具装箱) 和 P2900 (土地并购)，均为斜率优化 DP 模板题，代码实现规范。P4785 (交换) 代码展示了在树上进行状态记忆化搜索的能力。 | 已掌握斜率优化 DP 的原理与标准写法；能够处理有一定复杂度的树形结构与状态设计。                              |
| 输入输出/数值意识 | 稳      | 大量通过题目，未出现因数据范围或边界导致的大量失败。P3195 中 <code>L++</code> 的操作说明了对下标的精细控制。                    | 对常见数据范围（ <code>int</code> ， <code>long long</code> ）有清晰认知，能熟练处理输入输出边界。 |
| 动态规划 (DP) | 中上     | 已从基础 DP 进阶到斜率优化 DP。但难度 6 的题目仅有 4 道，刷题量集中在 DP 的中低难度区。                                  | 熟练掌握线性 DP、背包 DP、基础区间 DP 和经典的斜率优化 DP。                                   |
| 图论建模与算法   | 覆盖充分   | 难度分布图显示大量练习，但代码样本中缺乏图论题目。这表明图论是已扎实掌握的基础而非当前挑战点。                                       | 对最短路、最小生成树等基础图论模板及其变形的应用非常熟练。                                          |
| 贪心/构造/证明  | 中等偏稳   | 未在所有统计标签中特别突出，但高通过率暗示了在“直觉+试错”层面的贪心构造能力不错。P4785 的解法融合了贪心选择和 DP 状态。                    | 能够解决有明确贪心策略的题目，但在需要严格证明或极具创意的构造题上可能缺乏系统训练。                             |
| 计数与组合推导   | 明显短板   | 难度 5-6 的题仅占 11.2%，且无相关算法标签。计数问题通常集中在中高难度，刷题量的缺失与高阶 DP 能力不匹配。                          | 掌握了加法/乘法原理等基础组合知识。                                                     |

| 能力块    | 当前等级    | 数据证据                                                | 已经具备                       |
|--------|---------|-----------------------------------------------------|----------------------------|
| 字符串与数学 | 基础稳，高级弱 | 统计中无字符串相关算法偏好，也无难度 5+ 的典型数论/组合题。整体训练体系中这两块是“缺失的拼图”。 | 掌握基本的字符串处理、gcd、素数筛等基础数论知识。 |

2. 【考纲精准定级与知识点盲区】

- 当前对应等级水平
  - **GESP：已全面超越八年级标准。** 选手已掌握八年级所要求的图论算法（如斜率优化的前置单源最短路）和复杂 DP。
  - **NOI 大纲：正处于提高级向 NOI 级的过渡期。** 他已精通提高级的部分高级 DP 技巧，但在同级别的其他核心板块（计数、字符串、图论建模）存在断层，尚未形成进入 NOI 级的完整知识网络。
- 知识点强弱项
  - **掌握最好的 3 个考点：**
    - DP 优化** (提高级): 从代码样本看，对**斜率优化**的掌握达到实战水平。
    - 数据结构不变量** (提高级): 能够设计并维护较为复杂的 DP 状态（如 `map<int, int> f[N]`），理解了状态压缩/合并的核心思想。
    - 基础图论** (入门级/提高级): 大量的刷题训练使其对图的基础遍历、存储、最短路等有极深的理解和代码肌肉记忆。
  - **最薄弱的 3 个考点：**
    - 离散与组合数学** (提高级): **容斥原理、卡特兰数、二项式定理**。这是当前最致命的短板，直接导致在高难度比赛中遇到计数题就束手无策。
    - 图论建模变形** (提高级/NOI级): **二分图、网络流**。虽然基础图论扎实，但缺少将问题转化为二分图匹配或网络流模型的训练，这是通往 NOI 级的核心门槛。
    - 高级字符串** (提高级): **AC 自动机、后缀数组**。在统计中完全空白，是 NOI 级考察的重点之一。
- **训练盲区** 在当前“提高级”水平，选手**完全没有涉及/刷题数据中缺失**的必考知识点有：
  - **计数类 DP** (如求方案数、概率期望 DP)。
  - **二分图匹配** (匈牙利算法)。
  - **网络流** (Dinic 算法、最小割模型)。
  - **AC 自动机** (Trie 图、Fail 树)。

3. 【风险诊断与训练闭环表】

| 优先级        | 风险项        | 触发场景                                | 比赛症状                              | 根因判断                                             | 训练专题             | 验收标准                                             |
|------------|------------|-------------------------------------|-----------------------------------|--------------------------------------------------|------------------|--------------------------------------------------|
| S (高/立即处理) | 计数与组合数学缺失  | 题目要求输出对 $1e9+7$ 取模的方案数，而非最优值。       | 读不懂题，或在 DFS 暴力上浪费时间，无法抽象出组合模型。    | 训练体系偏向“求最优值”的系统性思维，缺乏对“统计对象集合”的方法论（容斥、DP计数）。     | 计数 DP 与容斥原理      | 30分钟内独立完成一道中等难度的计数类 DP，如洛谷 P1450。                |
| S (高/立即处理) | 高阶图论建模能力断层 | 题目背景为分配任务、路径不交、棋盘覆盖等。               | 感觉像图论题，但套不上最短/最小生成树模板，最终放弃或写错搜索。  | 不了解二分图、网络流等高级图论模型的“建图谱系”，缺少将问题映射到边、流、割的语义能力。     | 网络流与二分图匹配        | 1小时内独立完成一道网络流建模题，如洛谷 P2763，并正确解释建模含义。            |
| A (中/近期处理) | 贪心策略的正确性证明 | 遇到需要证明或极易反悔的贪心，或者按直觉写出的贪心总是 WA 一个点。 | 提交多次，每次修补一个漏洞，最终换题或靠特判蒙混过关。       | 习惯于从题解中获取贪心策略，缺乏自己证明“最优子结构”和“交换论证”的思维训练。         | 贪心证明法（交换论证/临项交换） | 能清晰口头证明 P2900 (土地并购) 中排序+斜率优化的贪心正确性，并解决一道新的贪心难题。 |
| A (中/近期处理) | DP 状态设计僵化  | 遇到非线性的、带环的、或在区间/树上需要合并复杂信息的 DP。     | 只会套用一维/二维线性 DP，面对树形依赖或复杂约束时，状态设计混 | 高水平难题（如 P4785）依赖记忆化搜索或非常规的 DP 设计，选手在此类题上经验少，成本高。 | 树形 DP 与状态压缩 DP   | 25分钟内 AC 一道经典的换根 DP 或状压 DP 题，如洛谷 P1879。          |

| 优先级              | 风险项            | 触发场景                    | 比赛症状                                 | 根因判断                             | 训练专题                  | 验收标准                                    |
|------------------|----------------|-------------------------|--------------------------------------|----------------------------------|-----------------------|-----------------------------------------|
|                  |                |                         | 乱，转移遗漏。                              |                                  |                       |                                         |
| <b>B (低/可后置)</b> | <b>高级字符串结构</b> | 题目涉及多模式串匹配、最长公共子串的极速求解。 | 只会用 Hash 或 KMP 硬刚，一旦超时或卡 Hash 就无计可施。 | 未进行过 AC 自动机、后缀自动机等高级结构的系统性学习与训练。 | <b>AC 自动机与 Fail 树</b> | 手写 AC 自动机模板并 AC 洛谷 P3808，并理解 Fail 树的概念。 |

优先级说明：S（高/立即处理）· A（中/近期处理）· B（低/可后置）。

#### 4. 【代码质量与工程习惯】

基于你通过的三段代码样本，我指出以下优点和坏习惯：

- **优点：**

**目标导向的写法：**在 P3195 和 P2900 中，斜率优化 DP 的代码非常精炼，`while` 循环和函数封装（如 `slope`）意图明确，说明你在写题时脑子里有清晰的数学模型，代码只是其翻译。

**合理的宏与封装：**使用 `#define ll long long` 避免了重复劳动，将斜率计算独立为函数，提高了可读性，这是竞赛级选手的优秀习惯。

- **必须改掉的坏习惯：**

**全局变量滥用：**三段代码都将 `n`、`f`、`sta` 等核心数组和变量定义为全局。在复杂题型（如树分治、分层图）中，这极易导致多组数据未清空或递归调用时变量相互污染。**必须养成将主要数据结构放在 `main` 或结构体里，通过参数传递的习惯。**

**缺乏调试痕迹与路径压缩：**P4785 的代码中，`//if(i==4)cout...` 这种被注释掉的调试输出还在。同时，记忆化搜索中 `if(f[i][a]==0)` 的判断不够健壮，如果状态值为 0 会导致重复计算。应该使用 `-1` 或一个标记函数 `count()` 来判断是否被访问。

**关键逻辑注释缺失：**P4785 的核心逻辑是一个复杂的 `if-else` 分支判断，用于处理最小值在三者中的不同位置。这段代码没有任何注释解释为什么这么分情况，三个月后你自己再看，绝对无法快速理解其正确性。**请在复杂逻辑块前，用一句话注释你的“不变量”或“分情况理由”。**

#### 5. 【定制训练题单】

这是一份为你定制的“**从提高级向 NOI 级跨越**”训练题单。目标是补齐核心短板。

| 题号/题型                            | 训练目标                                                       | 所属专题  |
|----------------------------------|------------------------------------------------------------|-------|
| P1450 [HAOI2010] 订货              | 学习 <b>计数 DP</b> ：在背包模型中计算方案数，处理至少/至多等限制条件。                 | 计数与组合 |
| P2606 [ZJOI2010] 排列计数            | 学习 <b>组合数学与树形DP的结合</b> ：理解小根堆性质如何转化为子树间的组合数问题。             | 计数与组合 |
| P3386 【模板】二分图最大匹配                | <b>重构图论模型认知</b> ：入门 <b>二分图匹配</b> ，掌握匈牙利算法，这是图论建模的基石。       | 图论建模  |
| P2763 试题库问题                      | <b>网络流建模实践</b> ：学习如何将“匹配”问题泛化为 <b>最大流模型</b> ，体会图论建模的威力。    | 图论建模  |
| P1972 [SDOI2009] HH的项链           | <b>数据结构维护不变量</b> ：学习 <b>离线处理/树状数组维护</b> “区间不同颜色数”的经典技巧。    | 能力进阶  |
| P3808 【模板】AC 自动机 (简单版)           | <b>补齐字符串短板</b> ：掌握 <b>AC 自动机</b> 的构建与查询，理解多模匹配的基本原理。       | 字符串   |
| P10978 【PR 3.0】十二】               | <b>强化贪心证明</b> ：这是一道需要通过 <b>临项交换法</b> 严格证明排序策略的题目，锻炼你的贪心内功。 | 贪心与构造 |
| P1879 [USACO06NOV] Corn Fields G | <b>巩固状压 DP</b> ：入门并强化 <b>状态压缩DP</b> ，这是解决小规模复杂约束问题的通用武器。   | 动态规划  |

## 6. 【未通过题目专属题解（从暴力到正解）】

由于你本次导出的记录中**没有未通过/卡住的题目**，这部分无法基于错误进行分析。这是一个非常优秀的状态，说明你对自己能力的边界有清晰的认知，或者在提交前已经解决了大部分问题。

为了发挥这部分的价值，我将以你训练题单中的 **P10978 [PR 3.0] 十二（贪心证明题）** 为例，演示一次完整的“暴力-观察-优化”解题思维过程，作为你未来自我训练的模板。

**题目：**P10978 [PR 3.0] 十二

- **AI 题解摘要：**核心在于通过“临项交换”推导出怪物的最优击杀顺序，并利用优先队列模拟过程。
- **a) 暴力思路怎么想？** 首先，题目的过程是顺序击杀，直觉是搜索所有可能的击杀顺序。
  - **状态表示：**可以爆搜所有  $n$  个数的全排列，复杂度是  $O(n! * n)$ 。
  - **部分分：**当  $n \leq 10$  时， $O(n!)$  的复杂度足以通过，可以拿到一小部分分数。



- **b) 瓶颈在哪里?** 时间卡在  $n$  会增大到  $1e5$  级别,  $O(n!)$  和  $O(2^n)$  的搜索都不可接受。我们必须找到一个确定性的顺序, 用一个  $O(n \log n)$  的算法得出答案。
- **c) 关键性质/不变量观察** 既然是贪心顺序, 我们就用**临项交换法**来推导。假设我们有一个击杀序列, 其中有两个相邻的怪物  $i$  和  $j$ 。我们在击杀它们之前, 攻击力为  $A$ , 已积累金钱为  $M$ 。我们比较两种顺序的优劣:

- **顺序  $(i, j)$ :**

- 杀死  $i$  后: 攻击力  $A_i = A + a_i$ , 金钱  $M_i = M + m_i$ 。
- 杀死  $j$  后: 攻击力  $A_{ij} = A_i + a_j = A + a_i + a_j$ , 得到报酬  $r_{ij} = m_j * A_i = m_j * (A + a_i)$ 。
- 此阶段总收益  $R1 = m_i * A + r_{ij} = m_i * A + m_j * A + m_j * a_i$ 。

- **顺序  $(j, i)$ :**

- 同理, 总收益  $R2 = m_j * A + m_i * (A + a_j) = m_j * A + m_i * A + m_i * a_j$ 。

我们希望总收益最大。比较  $R1$  和  $R2$ ,  $m_i * A + m_j * A$  是相同项, 忽略。我们只需要比较  $m_j * a_i$  和  $m_i * a_j$ 。若  $R1 > R2$ , 则  $m_j * a_i > m_i * a_j$ , 即  $a_i / m_i > a_j / m_j$  (假设  $m_i, m_j$  非0, 为0的情况同理可特判)。

**核心不变量:** 优先击杀“攻击力/报酬”比值更高的怪物, 即  $a/m$  更大的怪物, 整体收益更高。这符合直觉: 用更少的击杀成本 ( $m$ ) 换取更多的攻击力成长 ( $a$ )。

- **d) 最终正解的推导与核心代码结构**

**排序准则:** 按照我们推导出的  $a / m$  比值对所有怪物从大到小排序。这是固定的全局最优击杀顺序! 但是, 我们的攻击力在击杀过程中会变, 这会影响收益计算, 排序准则只是规定了决策的优先级。

**过程模拟:** 现在我们有一个最优顺序, 但还有一个问题: 怪物会逃跑。一个怪物  $i$  只有在  $m_i$  小于等于当前金钱数时才能被击杀。如果当前没钱击杀最优的怪物怎么办?

**反悔机制:** 我们按排序顺序遍历怪物, 尝试用当前金钱击杀。

- 如果当前金钱足够 ( $money \geq monster.m$ ), 果断击杀! 获得金钱和攻击力。
- 如果当前金钱不够, 我们看看之前已击杀的怪物中, 有没有一个  $m$  值比当前怪物的  $m$  值更大的“昂贵”怪物?
  - 如果有, 我们“反悔”: 我们用当前这个更便宜但  $a$  值更优的怪物, 替换掉那个最昂贵的怪物。将这个最贵的怪物“踢出”已击杀列表, 此时金钱会立刻恢复 ( $money += expensive\_monster.m$ ), 然后我们就有足够钱击杀当前怪物了 ( $money -= current\_monster.m$ )。

**数据结构:** 为了高效地找到“已击杀怪物中  $m$  值最大的”, 我们用一个**最大堆 (优先队列)**来维护已击杀怪物的  $m$  值。

**核心代码结构：** ``cpp // 1. 定义结构体与排序 struct Monster { int a, m; bool operator<(const Monster& o) const { // 核心：按 a/m 降序排序，注意处理正负和零 return 1LL \* a \* o.m < 1LL \* o.a \* m; } }; sort(monsters.begin(), monsters.end());

// 2. 模拟与反悔 long long cur\_atk = initial\_atk; long long cur\_money = 0; priority\_queue<int> killed\_m\_pq; // 大根堆存已杀怪物的m值

```
for (auto& mon : monsters) { if (cur_money >= mon.m) { // 钱够，直接杀 cur_money -= mon.m; cur_atk += mon.a; killed_m_pq.push(mon.m); } else if (!killed_m_pq.empty() && killed_m_pq.top() > mon.m) { // 钱不够，但可以反悔，踢掉一个最贵的 int expelled_m = killed_m_pq.top(); killed_m_pq.pop(); cur_money += expelled_m; // 恢复金钱 // 注意：踢出的怪物提供的攻击力也要扣掉，这需要更多状态维护 // 这部分留给你继续思考，完整解法需要同时维护攻击力。 } }
```

**最终提示：**这道题的反悔机制维护比这里写的要复杂，因为踢出一个怪物也需要扣掉它的攻击力，这导致我们需要维护的不是简单的 `m` 的最大堆，而是结合了 `a` 和 `m` 的更复杂的堆结构。这个推导过程完美展示了如何从零开始，一步步通过观察和证明，将一个看似复杂的贪心问题，转化为有明确数学性质和数据结构模型的过程。祝你好运，教练相信你有能力攻克它！